

Final Exam: Room 314

A Spooky Text Adventure Game

Joe Ording

Project Summary

- Direction-based exploration with random events and environment.
- Multi-step puzzle path with randomized clues.
- Wrong turns loop the player back with fresh clues.
- Environmental storytelling through environmental descriptions, events, and puzzles.
- Inventory system: player has a watch, backpack, phone, keys, and wallet.
- Split-pane UI with live room/event panels.
- Multiple outcomes based on time remaining; clock is always ticking down to when your final starts.

```
FINAL EXAM: ROOM 314                                     Time: 10:00
LOCATION
Building Lobby

DETAILS
The lobby is almost empty. The hallway straight ahead is blocked off with orange cones and caution
tape - construction, apparently. A hand-lettered detour sign points down a narrower hallway to your
left that you've never noticed before. Behind you, the main exit doors are chained shut. The
fluorescent lights buzz at a frequency just below comfortable.

EXITS
forward, left, back

YOU NOTICE
a hand-lettered detour sign

ACTIVITY

ROOM_314
FIND YOUR
CLASSROOM

FINAL EXAM: ROOM 314
Your final exam starts in 10 minutes. Find Room 314.
Type HELP for a list of commands.
```

Core Gameplay Loop

1. Read the randomized clue at the four-way intersection, go the correct way down randomized hallway.
2. Reach the bathroom junction and solve the handwashing puzzle, get clue.
3. Use the clue to find the janitor intersection.
4. Listen to the janitor's clue for the final direction, proceed down randomized hallway.
5. Reach Room 314 before the timer expires for a 'good' ending.

```
FINAL EXAM: ROOM 314                                     Time: 8:45
LOCATION -
4-Way Intersection

DETAILS
A four-way intersection. Identical hallways stretch in every direction. The fluorescent light above flickers twice, then steadies.

A bright orange flyer is taped to the wall: "CS CLUB BAKE SALE - THIS WAY -" with a hand-drawn arrow pointing forward.

EXITS
forward, back, left, right

ACTIVITY
FIND YOUR CLASSROOM

FINAL EXAM: ROOM 314
Your final exam starts in 10 minutes. Find Room 314.
Type HELP for a list of commands.

> left
You check your phone. No signal. The battery shows 3%.
> forward
> forward
A door closes somewhere down the hall. You didn't see anyone.
> forward
> forward
You hear footsteps behind you. When you turn around, no one is there.
```

Architecture Overview

- `main.py` is the root: it loads data, builds the world, registers commands, and selects the UI.
- `data/\*.yaml` holds player-facing narrative text, room definitions, commands, events, puzzle clues, etc. - nothing hard-coded.
- `game/` is split by responsibility: command parsing, movement, puzzles, rendering, state, world loading, and engine flow.

```
main.py > ...
1 """Composition root for Final Exam: Room 314."""
2
3 from __future__ import annotations
4 from game.engine import GameEngine
5 from game.event import load_events
6 from game.player_commands import build_commands
7 from game.state import GameState
8 from game.world import build_world
9
10
11 def main() -> None:
12     """Build the game world and start the default engine.
13
14     Command-line argument parsing is kept in this composition root so the
15     repository's public entry point remains ``python main.py``.
16     """
17     import argparse
18
19
20 data > ! rooms.yaml > ...
1 # rooms.yaml - all room definitions for Final Exam: Room 314
2
3 # Each entry has:
4 #   room_id - unique string key
5 #   name - short display name
6 #   description - full prose description
7 #   exits - map of direction -> destination room_id (null = blocked)
8 #   items - optional map of item_id -> display name
9 #   attributes - optional map of arbitrary game-logic keys/values
10 #   type - "structural" or "flavor" (used to build FLAVOR_ROOM_POOL)
11
12 # Exit values that are set at runtime by puzzle logic are written as null here.
13 # Flavor-room forward/back exits are also null; routing wires them at runtime.
14
15 rooms:
16   # — structural rooms —
17
18   - room_id: lobby
19     name: Building Lobby
```

Agentic Coding Approach

- Tailor agent rules and skills for each project defining the language(s), tools to use, and other specifics of the architecture.
- Work is driven from 'README.md' and 'TODO.md', to track project status and pending/in progress tasks.
- Documentation kept in sync with implementation through 'DEVELOPER.md', 'ARCHITECTURE.md', and 'CHANGELOG.md' updates alongside code changes.
- The goal is maintainability: strict docstring compliance, avoid overlapping helpers, clear module ownership, and no 'legacy' layers left behind.

```
.github > prompts > task.prompt.md
1  ---
2  agent: agent
3  description: "Continue Current Task Workflow (TODO-driven)"
4  ---
5
6  > NOTE: For large, repo-wide code hygiene or analysis work (symbols, call graph, orphans,
7  > duplicates, shared modules), prefer the '/code' agent prompt and follow
8  > '.github/AGENT_WORKFLOW.md'. Use this '/task' prompt for general feature/bug/maintenance
9  > tasks anchored in 'TODO.md'.
10
11 1. Identify & Lock the Active Task
12   - Open 'TODO.md'.
13   - Find the very first top-level checklist item ('[ ]' or '[/]').
14   - Reply:
15
16     | > Working on: '[/]' under '<parent heading>'
17
18   - Prepend '<!-- working-on: <task text> -->' at the top of 'TODO.md' and mark that task as
19     '[/]' (in progress).
20   - Then immediately move on--do not wait for confirmation.
```

Testing Strategy

- 'make test' runs the full unittest suite across command handling, puzzle flow, world building, endings, events, and curses helpers.
- 'make coverage' enforces an 80% statement-coverage floor for 'main.py' and 'game/*.py'.
- Tests are split by responsibility, with focused helper suites and end-to-end flow coverage.
- Refactors were validated with narrow tests first, then the broader project commands.

```
Run with unittest | Run with pytest
def test_current_room_view_only_adds_reset_line_after_return(self) -> None:
    """Verify that the 4-way reset line only appears after the player has returned there."""
    engine, _room = make_intersection_engine()

    first_view = engine.current_room_view()
    assert first_view is not None
    self.assertNotIn("feels like a reset", first_view.description)

    engine.state.set_flag("step1_returned_to_intersection")

    returned_view = engine.current_room_view()

    self.assertIsNotNone(returned_view)
    assert returned_view is not None
    self.assertIn("feels like a reset", returned_view.description)

Run with unittest | Run with pytest
def test_current_room_view_uses_shared_janitor_hint_helper(self) -> None:
    """Verify that the janitor hallway clue comes through the shared janitor formatter."""
    engine = make_engine(start_room="hallway_janitor", time_remaining=240)
    engine.state.active_clues["step3_song_chorus"] = (
        "Take the left hall\nTake it again\nOne more line"
    )
```

Key Takeaways

- YAML-based narrative and gameplay text content separate from Python - codebase is more modular and reusable, easier to debug, able add/change content without touching code.

- Randomized clues and interstitial hallways make the experience replayable/expandable while staying testable.

- The final codebase emphasizes documentation, modular ownership, and regression coverage.

<https://github.com/ordingj/CS115-F26-Group-Project>

THANK YOU